UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE
AND TECHNOLOGY

SCHOOL OF SCIENCE

DEPARTMENT OF MATHEMATICS

Option: MATHEMATICS AND STATISTICS

FINAL YEAR PROJECT REPORT

Optimizing maize distribution logistics in Rwanda using the Minimum Cost Capacitated Flow Problem (MCCFP)

A case study of Eastern Province and Kigali City through East Africa Exchange (EAX) warehousing infrastructure

PROJECT NUMBER :

Prepared by:

Names	**Reg Number**
Jean de la Croix Shimimana	222009803
Esther Mahoro	222011181

SUPERVISOR: Felix Ndayambaje

ACADEMIC YEAR: 2024/2025

I. Declaration

We, Jean de la Croix Shimimana (222009803) and Esther Mahoro (222011181), students from the University of Rwanda, College of Science and Technology (CST), School of Science, Department of Mathematics.

The undersigned, hereby declare that this research study entitled "**Optimizing maize distribution logistics in Rwanda using the Minimum Cost Capacitated Flow Problem (MCCFP): A case study of Eastern Province and Kigali City through East Africa Exchange (EAX) warehousing infrastructure**" under the supervision of Felix Ndayambaje is our original work and has never been presented or submitted by someone else for any academic award in any university or institution of higher learning and that any work done by others or by ourselves previously has been acknowledged and referenced accordingly.

Signed by: .....

Date: ...../...../2025

II. Dedication

We dedicate this thesis to our families and loved ones, whose unwavering support and encouragement have been our foundation throughout this journey.
We also dedicate it to each other—for the teamwork, perseverance, and shared commitment that brought this work to life.

III. Certification

This is to certify that the study entitled "**Optimizing maize distribution logistics in Rwanda using the Minimum Cost Capacitated Flow Problem (MCCFP): A case study of Eastern Province and Kigali City through East Africa Exchange (EAX) warehousing infrastructure**" is a record of an original work done by Jean de la Croix Shimimana (222009803) and Esther Mahoro (222011181) for the partial fulfilment of the requirements of the award of bachelor's degree with Honour in Mathematics and Statistics in the University of Rwanda, College of Science and Technology (CST) during the academic year 2024/25.

Supervisor: Head of Mathematics Department:

Date…... /...../2025 Date: ...../…../2025

IV. Acknowledgement

We are deeply grateful for the support and guidance we received throughout the course of our final year thesis. Working together as a pair has been a rewarding experience, allowing us to complement each other's strengths, share responsibilities, and grow both academically and professionally.

We would like to sincerely thank our supervisor, Felix Ndayambaje, for their continuous support, insightful feedback, and encouragement throughout this journey. Their expertise and mentorship played a vital role in shaping the direction and outcome of our work.

We also wish to thank the Department of Mathematics, University of Rwanda – College of Science and Technology, for providing us with the academic environment and resources necessary to complete this thesis.

Finally, we extend our appreciation to our families, friends, and classmates for their moral support and encouragement during this important phase of our academic journey.

V. Abstract

Efficient distribution of staple crops is essential to ensuring food security and reducing post-harvest losses, especially in developing countries like Rwanda. The Eastern Province produces a surplus of maize, yet high transportation costs and underutilized warehousing infrastructure contribute to inefficient supply chains and inconsistent delivery to urban areas such as Kigali City. This project addresses these logistical challenges by applying the **Minimum Cost Capacitated Flow Problem (MCCFP)**, a mathematical optimization model that minimizes transport costs under real-world constraints such as truck capacity, warehouse limits, and district-level supply and demand.

Using East Africa Exchange (EAX) warehouses as intermediate hubs, the model simulates maize flows between supply zones in Eastern Province and demand zones in Kigali City. The objective is to determine the lowest-cost distribution strategy that fully satisfies maize demand while making effective use of the EAX storage network. The model is formulated as a linear programming problem and solved using the **Network Simplex Method**, implemented in Python with libraries such as PuLP, SciPy, and NetworkX. Inputs were drawn from national agricultural statistics, population data, and cost estimates, while some values were simulated where direct records were unavailable.

Results from the model demonstrate a significant reduction in overall distribution costs compared to uncoordinated delivery practices. Moreover, the optimization revealed that several high-production districts were excluded due to long distances, and that some warehouses were underused despite large capacities highlighting the trade-off between storage availability and transport efficiency. These findings suggest that incorporating mathematical planning tools like MCCFP into agricultural logistics can help Rwanda make better use of its existing infrastructure and inform more strategic decisions for future investments.

VI. Table of contents

I. Declaration ..... **ii**
II. Dedication ..... **iii**
III. Certification ..... **iv**
IV. Acknowledgement ..... **v**
V. Abstract ..... **vi**
VI. Table of contents ..... **vii**
VII. List of figures ..... **ix**
VIII. List of tables ..... **ix**
IX. List of abbreviations and acronyms ..... **ix**
1. INTRODUCTION **2**
1.1 Problem Statement 2
1.2 Objectives of the Study 2
1.3 Scope of the Study 3
2. LITERATURE REVIEW **3**
2.1 Introduction 3
2.2 Agricultural Logistics and Distribution 3
2.3 MCCFP and Its Use in Logistics Optimization 4
2.4 Maize Distribution: Global to Local Contexts 4
2.5 Warehousing and Structured Trade Platforms in Rwanda 5
2.6 Research Gap 5
3. METHODOLOGY **5**
3.1 Data collection and assumptions 5
3.2 Network representation 6
3.3 Mathematical formulation of the Minimum Cost Capacitated Flow Problem (MCCFP) 6
3.4 Optimization Algorithm: The Network Simplex Method 7
3.5 Illustrative Example: Maize Flour distribution network 11
4. RESULTS, PRESENTATION AND ANALYSIS **18**
4.1 Problem definition and Data Integration: 18
4.2 Model implementation and solution 22
4.3 Key insights 22

4.4 Visualizations 23
5. DISCUSSION, CONCLUSION & Recommendations 25
5.1 Discussion 25
5.2 Conclusion 26
5.3 Recommendations 26
6. References 27
7. Appendix 28

VII. List of figures

a. Fig1.: Maize Flour distribution network
b. Fig2.: Iteration 0 | MCCFP illustrative example
c. Fig3.: Iteration 1| MCCFP illustrative example
d. Fig4.: Iteration 2| MCCFP-illustrative example
e. Fig5.: Optimal Solution| MCCFP illustrative example
f. Fig6.: Initial arc connections for this project
g. Fig7.: Python MCCFP solver for this project
h. Fig8.: Optimal solution arc connections
i. Fig9.: Supply utilization per district
j. Fig10.: Demand Fulfillment per District

VIII. List of tables

Table1.: Dataset used in this project

IX. List of abbreviations and acronyms

a. **MCCFP**: Minimum Cost Capacitated Flow Problem
b. **EAX**: East Africa Exchange
c. **CST**: College of Science and Technology
d. **UR**: University of Rwanda
e. **Rwf**: Rwandan Francs
f. **NISR**: National Institute of Statistics of Rwanda
g. **MINAGRI**: Ministry of Agriculture and Animal Resources
h. **RURA**: Rwanda Utilities Regulatory Authority
i. **GIS**: Geographic Information System
j. **LP**: Linear Programming
k. **IDP**: Inter-District Planning
l. **GDP**: Gross Domestic Product
m. **FAO**: Food and Agriculture Organization
n. **UN**: United Nations
o. **km**: Kilometer
p. **ton**: Metric Ton (1,000 kilograms)

1. INTRODUCTION

Maize is one of the most widely grown and consumed staple crops in Rwanda. It plays a crucial role in food security and local trade, with large production volumes in the Eastern Province and rising demand in urban centers like Kigali City. However, challenges such as poor road conditions, high transport costs, and underused storage infrastructure make it difficult to move maize effectively.

This study proposes the application of the Minimum Cost Capacitated Flow Problem (MCCFP), a structured mathematical approach to minimizing logistics costs while considering real-world constraints like truck capacity, storage limits, and regional supply-demand variations. It is particularly useful in agricultural contexts, where transportation costs directly affect farmer profitability and food accessibility.

By applying this model to maize transportation from the Eastern province to Kigali city, and treating EAX warehouses as transshipment points, the study aims to offer a more efficient logistics framework tailored to Rwanda's specific context.

1.1 **Problem Statement**

Rwanda's Eastern province districts produce a surplus of maize, but poor distribution planning leads to supply-demand imbalances, especially in high-consumption areas like in the districts of Kigali city. Transport routes are often uncoordinated and costly, limiting profits for farmers and raising prices for consumers. Although EAX warehouses were set up to support efficient storage and trade, they are underused due to the lack of a centralized logistics system. This study aims to address these challenges by applying the Minimum Cost Capacitated Flow Problem (MCCFP) to optimize maize movement, reduce costs, and improve overall distribution efficiency.

1.2 Objectives of the Study

General objective:

To optimize inter-district maize distribution in Rwanda using the MCCFP framework, minimizing logistical costs while satisfying demand and respecting warehouse and transport constraints.

Specific Objectives:

1. To model maize distribution as a capacitated flow problem across selected districts of Eastern province and Kigali city.

2. To incorporate EAX-certified warehouse capacities, truck constraints, and maize demand/supply quantities into the Minimum Cost Capacitated Flow Problem (MCCFP) model.
3. To compute the optimal distribution flows using the network simplex algorithm.
4. To obtain the optimized distribution pattern, compare it with conventional patterns and highlight areas for operational improvement.

1.3 Scope of the Study

- **Geographic scope:** This study focuses solely on the districts of Eastern Province (Supply region) and those of Kigali city (Demand region) of Rwanda.
- **Operational scope:** The study covers maize distribution between districts, assuming shared maize collection points per district, and intermediary EAX-certified warehouses available in the region.
- **Methodological Framework:** The study employs the Minimum Cost Capacitated Flow Problem (MCCFP) as its primary analytical model. This approach is structured as a linear programming problem and resolved using the Network Simplex Method. The implementation was conducted in Python, utilizing optimization libraries tailored for network flow modeling.
- **Data Considerations**: The model relies on simulated demand and supply data assembled by a secondary data collection approach referring to various statistical reports by accredited bodies in Rwanda.
- **Optimization Objective:** The main goal is to minimize the total annual maize distribution cost.

2. LITERATURE REVIEW

2.1 Introduction

This chapter reviews key literature on agricultural logistics, supply chain optimization, and mathematical modeling using the Minimum Cost Capacitated Flow Problem (MCCFP). It also situates the study within global, regional, and local contexts, highlighting both the importance of maize and the need for improved distribution models in sub-Saharan Africa and Rwanda.

2.2 Agricultural Logistics and Distribution

Efficient movement of agricultural products is essential for reducing losses, stabilizing prices, and improving food security. The Food and Agriculture Organization (FAO, 2023) reports that approximately 13% of food produced globally is lost before it reaches consumers, largely due to

poor transport systems, weak storage, and lack of coordination between actors in the supply chain. Many countries have adopted logistics planning tools such as linear programming and network optimization to address these issues (Klibi & Martel, 2012; Grote et al., 2021). These tools have proven effective in minimizing costs and improving coordination, particularly in well-developed markets.

2.3 MCCFP and Its Use in Logistics Optimization

The Minimum Cost Capacitated Flow Problem is a type of linear programming model designed to minimize transportation costs while satisfying constraints such as supply limits, demand requirements, and capacity restrictions. MCCFP has been used in a wide range of sectors, from vaccine distribution and humanitarian aid to electricity supply networks.

For example, Chen et al. (2020) applied the model to vaccine delivery systems, while Balcik and Beamon (2008) used it in emergency logistics during humanitarian crises. Ghamkhari et al. (2015) used it to optimize the flow of electricity through power grids with limited transmission capacity. Despite these diverse applications, the model has rarely been applied in agriculture, especially in the African context. This study aims to explore its potential in optimizing maize logistics in Rwanda.

2.4 Maize Distribution: Global to Local Contexts

Globally, maize ranks among the top staple crops. Countries like the U.S., China, and Brazil have built advanced logistics networks that support large-scale production and export. These systems depend on well-developed roads, storage infrastructure, and digital coordination platforms. By contrast, many low-income countries continue to struggle with fragmented distribution systems, often resulting in high post-harvest losses and volatile prices.

In sub-Saharan Africa, maize is both a key food source and a central part of rural economies. However, weak transport infrastructure, limited storage options, and the dominance of informal trade channels create major challenges. These issues increase food prices, reduce farmer earnings, and worsen food insecurity, particularly during harvest or drought seasons (Santpoort, 2020).

Within East Africa, maize is a major regional trade commodity. Countries such as Kenya, Uganda, and Tanzania export and import maize depending on seasonal surpluses and deficits. Efforts to streamline cross-border trade have had some success, but internal distribution networks remain underdeveloped. Informal trade still dominates, and structured storage or transport systems are underutilized.

In Rwanda, maize is mainly grown in the Eastern Province and transported to Kigali. Despite strong production levels, farmers often face challenges storing or moving their harvest

efficiently. Many sell their maize quickly after harvest at low prices due to lack of access to storage or structured markets. The certified EAX warehouses, although present, are rarely used to their full capacity, often due to limited awareness and lack of integration into transport planning.

This study addresses these gaps by applying MCCFP to simulate and optimize maize movement leveraging EAX warehouses as optional transshipment points. The goal is to create a system that balances cost, supply, demand, and infrastructure, offering a scalable logistics solution.

2.5 Warehousing and Structured Trade Platforms in Rwanda

To improve grain distribution, Rwanda has invested in certified warehousing infrastructure, especially through the EAX. These platforms were introduced to promote structured trade, improve price transparency, and reduce spoilage. Research shows that when warehouses are linked to broader logistics planning, they can improve delivery efficiency and reduce transport costs (Cox et al., 2020). However, in practice, these systems remain underutilized due to lack of integration with transport and market data. This study explores how better use of these facilities within an optimization model can help strengthen maize supply chains.

2.6Research Gap

Although MCCFP has been widely used in health and infrastructure sectors, its application in agricultural logistics is still limited. Most studies on maize in Rwanda focus on production, pricing, or food security, not on how to move it efficiently from rural to urban areas using structured models. Moreover, while EAX warehouses exist, little research has been done on how to include them in a national distribution framework. This study fills that gap by integrating warehousing and cost minimization into a single model that can guide real-world maize distribution planning.

3. METHODOLOGY

The Eastern province, with fertile land and large maize farms, serves as Rwanda's primary maize surplus zone. Contrary, Kigali being a highly urbanized city, is a net consumer with minimal agricultural output. The **East Africa Exchange (EAX)** operates warehouses in the Eastern province, facilitating harvest aggregation, storage and trade. This study focuses on optimizing maize transportation from surplus districts to deficit ones leveraging the EAX infrastructure as an additional infrastructure that can streamline the maize distribution patterns.

3.1 Data collection and assumptions

Secondary data sources

- **Supply data:** Derived from NISR's 2024 Seasonal Agriculture Survey, summing production quantities from season A and B per district.
- **Demand data:** calculated using the average per capita maize consumption (35.42 kg/year from 2017-2021 Food balance sheet by NISR and district population data from the 5th Rwanda population and housing census (2022).
- **Warehouse capacities:** Pulled from official EAX website.
- **Transportation cost Estimate:** Assumed 200 RWF/km/ton as an average rate, estimated from RURA and MINAGRI-referenced cost structures.
- **Distances:** Measured using google maps between inter districts shared collection points or warehouse locations.
- **Truck assumptions:** A 20-ton capacity truck, carrying out 4 trips per week was assumed, summing up to a total of 208 trips per year per truck.
- **Lower trip capacity:** Minimum revenue per trip was set at 10,000 Rwf and this amount helped to determine the minimum tons/trip based on the total cost/trip.
- **Upper trip capacity:** Determined by maximum feasible movement of maize considering supply, demand and truck trips.

3.2 Network representation

The maize distribution network is modeled as a directed graph $G = (N, A)$ where:

- N= Set of nodes including supplies (Eastern province districts), demands (Kigali city districts) and transshipment nodes (EAX warehouses).
- A= Set of arcs: each arc (i, j) represents a possible route for maize transport from node I to node j.
- Each arc is defined by the following parameters
  X_{ij}: quantity of maize in tons transported from node i to node j.
  U_{ij}: Upper capacity limit of arc, defining the maximum maize quantity that can flow through a specific connection
  L_{ij}: Lower capacity limit of arc, defining the minimum maize quantity that can flow through a specific connection
  C_{ij}: Unit cost of transportation along arc (i, j)
  F_i: Net flow at node i: positive for supply nodes, negative for demand nodes and zero for intermediate (transshipment) nodes like EAX warehouse.

This formal network structure serves as a mathematical foundation for evaluating the current system and proposing optimization strategies.

3.3 Mathematical formulation of the Minimum Cost Capacitated Flow Problem (MCCFP)

Modeling assumptions

The MCCFP operates under standard assumptions namely:

- Each arc (i, j) in the network has a non-negative unit transportation cost C_{ij} , representing the cost of moving one ton of maize from node i to node j.
- Arcs have **upper and lower capacity limits**. They represent respectively the minimum and maximum tons a truck moving from i to j should have.
- Each node i has a net flow value F_i , positive for the supply node (Eastern province districts) negative for demand nodes (Kigali city districts), and zero for intermediate or transshipment nodes (EAX warehouses).

Objective function

The goal is to **minimize total annual maize distribution cost**

Minimize $\boldsymbol{Z}$= $\sum_{(i,j)\in E} C_{ij} \cdot X_{ij}$

Where:

- $X_{ij} =$ flow of maize (in tons) along the arc (i, j)
- $C_{ij} =$ unit cost (Rwf/Ton) on arc (i, j)
- $E =$ Set of arcs in the network

Constraints

1. Flow Conservation constraints:

$$\sum_{j:(i,j)\in E} X_{ij} - \sum_{k:(k,i)\in E} X_{ki} = F_i \; Where \begin{cases} F_i = S_i \text{ if } i \text{ is a supply node} \\ F_i = -D_i \text{ if } i \text{ is a demand node} \\ F_i = 0, for\ transshipment\ nodes \end{cases}$$

2. Capacity Constraints:

For each arc (i, j): $L_{ij} \leq X_{ij} \leq U_{ij}, \forall (i, j) \in E$

Where C_{ij}, the capacity of the arc (i, j).

3. Non-Negativity:

$X_{ij} \geq 0, \forall (i, j) \in E$

3.4 Optimization Algorithm: The Network Simplex Method

To solve the Minimum Cost Capacitated Flow Problem (MCCFP), this study employs the Network Simplex Method- a specialized algorithm that efficiently handles large-scale network flow problems. Unlike the general simplex method for linear programming, the network simplex leverages the unique structure of flow networks, making it both faster and more intuitive for distribution logistics. Its efficiency stems from the fact that all basis matrices in network flow

problems are totally unimodular, ensuring that all basic feasible solutions have integer values if the supply/demand and capacities are integers.

Core Algorithmic Framework

The Network Simplex Method operates on a fundamental principle: a basic feasible solution in a network flow problem always corresponds to a spanning tree of the network. A **spanning tree** is a subset of the network's arcs that connect all nodes without forming any cycles, and it always contains exactly n-1 arcs, where n is the number of nodes.

A crucial assumption for the Network Simplex Method is that the network must be **balanced**, meaning the total supply equals the total demand. Mathematically, this is expressed as:

$$\sum_{i \in N} F_i = 0$$

The algorithm iteratively moves from one basic feasible solution (one spanning tree) to another, systematically improving the total cost until an optimal solution is reached. The arcs in the spanning tree are called basic arcs, and all other arcs are nonbasic arcs. Nonbasic arcs are typically at their lower bound or upper bound.

Key Elements for Network Simplex Success

The success and efficiency of the Network Simplex Method relies on several key elements that simplify the general simplex operations:

1. **Spanning Tree Structure:** The algorithm maintains a spanning tree throughout its iterations. This structure simplifies basis computations, as all basic variables (flows in the spanning tree arcs) can be uniquely determined once the dual variables are known.

2. **Dual Variables ($\mathbf{W_i}$):** For each node i in the network, a dual variable $\mathbf{W_i}$ **(**also known as a node potential or price) is associated. These dual variables are crucial for determining the "cost" of sending flow between nodes and for identifying potential improvements.

    - For any basic arc (i, j) in the spanning tree, the dual variables satisfy the equation:
    $$W_i - W_j = C_{ij}$$
    Where C_{ij} is the unit cost of flow on arc (i, j). Since there are n nodes and n-1 basic arcs, we can arbitrarily set one dual variable (e.g., $\mathbf{W_1 = 0}$ and then solve for the remaining n-1 dual variables using the n-1 basic equations.

3. **Reduced costs ($\mathbf{Z_{ij} - C_{ij}}$):** The value $\mathbf{Z_{ij} - C_{ij}}$ indicates how much the total cost would change if one unit of flow were to be sent through the nonbasic arc (i, j).
Opposed to the usual C_{ij} which expresses the actual direct unit cost of sending one unit of flow from node i to node j along the arc connecting them, Z_{ij} represents the implicit cost

of sending one unit of flow from node i to node j through the existing basic arcs (i.e., by traversing the unique path in the basic spanning tree that connects i and j).
The difference, $Z_{ij} - C_{ij}$ quantifies the net change in the total objective function cost if one unit of flow were to be introduced into the nonbasic arc (i, j).

- The reduced cost for a nonbasic arc (i, j) is calculated as:

$$\mathbf{Z_{ij} - C_{ij} = W_i - W_j - C_{ij}}$$

- If a nonbasic arc is at its lower bound and has a positive reduced cost $(\mathbf{Z_{ij} - C_{ij} > 0})$**,** it means increasing flow on this arc would decrease the total cost. This arc is a candidate to enter the basis.
- If a nonbasic arc is at its upper bound and has a negative reduced cost $(\mathbf{Z_{ij} - C_{ij} < 0})$**,** it means decreasing flow on this arc (i.e., sending flow in the reverse direction) would decrease the total cost. This arc is also a candidate to enter the basis.
- If all nonbasic arcs satisfy the optimality conditions (positive reduced cost for arcs at upper bound, non-positive for arcs at lower bound), the current solution is optimal.

4. **Cycle Formation and Flow Adjustment:** When a nonbasic arc is chosen to enter the basis, adding it to the current spanning tree creates a unique cycle. The algorithm then adjusts the flow around this cycle to introduce flow on the entering arc while maintaining flow conservation at all nodes and respecting arc capacities.

Algorithmic Steps

The Network Simplex Method proceeds in the following steps:

- **Step 0: Determine an Initial Basic Feasible Solution**
  The first step is to identify a starting basic feasible solution. This involves finding a set of n-1 arcs that form a spanning tree, satisfying all flow conservation constraints at each node and respecting the lower and upper capacity limits of all arcs.
  Practical Approaches: For small problems, an initial solution can sometimes be found by inspection. For larger or more complex networks, techniques such as the "Northwest Corner Rule" (adapted for networks) or the "Artificial Variable Technique" (similar to the general simplex method) can be employed to systematically construct an initial basic feasible solution. The goal is to ensure that all demands are met, supplies are distributed, and no arc capacity is violated.
- **Step 1: Determine an Entering Arc (Optimality Condition)**
  This step checks if the current basic feasible solution is optimal.
  - **Compute Dual Variables ($\mathbf{W_i}$):**
    a. Arbitrarily set one dual variable, say $\mathbf{W_1}$=0.

b. For every basic arc (i, j) in the current spanning tree, use the relation $W_i - W_j = C_{ij}$ to solve the remaining n-1 dual variables. Work outwards from the node whose dual variable was set to zero.

- **Compute Reduced Costs ($\mathbf{Z_{ij} - C_{ij}}$):**
  a. For every nonbasic arc (i, j) (arcs not in the spanning tree), calculate its reduced cost using the formula: $\mathbf{Z_{ij} - C_{ij} = W_i - W_j - C_{ij}}$.

  Where: $\boldsymbol{Z_{ij} = W_i - W_j}$ represents the cost implied by the node potentials $(\boldsymbol{W_i - W_j})$ if flow were to pass through arc (i, j).

- **Select entering Arc:**
  a. If all nonbasic arcs satisfy the optimality conditions (i.e., $\mathbf{Z_{ij} - C_{ij} \leq 0}$ for arcs at their lower bound, or $\mathbf{Z_{ij} - C_{ij} \geq 0}$ for arcs at their upper bound), then the current solution is optimal, and the algorithm terminates.
  b. Otherwise, select a nonbasic arc that violates the optimality condition. Typically, the arc with the most positive reduced cost (if it's at its lower bound) or the most negative reduced cost (if it's at its upper bound and flow needs to decrease) is chosen as the **entering arc.** This arc offers the greatest potential for improving the objective function.

➢ **Step 2: Determine the Leaving Arc (Feasibility Condition) and Update the Solution**
Once an entering arc is identified, it is added to the current spanning tree. This action creates a unique cycle in the network.

- **Identify the Cycle:** Trace the path from the "tail" of the entering arc to its "head" using only existing basic arcs and then close the loop by adding the entering arc.
- **Determine Flow Adjustment (Δ):**
  a. Identify arcs in the cycle where flow will increase (in the same direction as the entering arc) and arcs where flow will decrease (in the opposite direction).
  b. Calculate the maximum possible flow change, Δ. This is limited by:
     i. The remaining upper capacity of arcs where flow is increasing
     ii. The current flow (above lower bound) of arcs where flow is decreasing.
  c. The smallest of these limiting values determines Δ.
- **Select Leaving Arc:** The arc that reaches its capacity limit (either its upper bound or its lower bound of 0) first, because of the Δ flow adjustment, is the leaving arc. This arc is removed from the basis.
- **Update flows and Basis:**

a. Adjust the flows on all arcs in the cycle by Δ.

b. The entering arc replaces the leaving arc in the spanning tree, forming a new basic feasible solution.
c. Return to Step 1.

This iterative process continues, moving from one spanning tree to another, until the optimality conditions are met, yielding the minimum-cost flow.

3.5 Illustrative Example: Maize Flour distribution network

A logistics network connects two maize processing plants to two major cities. The monthly production capacity at the two plants is 40 and 50 thousand tons of maize flour, and the monthly demand from the two cities is 30 and 60 thousand tons. Nodes 1 and 2 represent Plant 1 and Plant 2, and nodes 4 and 5 represent City 1 and City 2. Node 3 is a central distribution warehouse, which serves as a transshipment point for maize flour between the plants and the cities. The model is already balanced because the total supply from the plants equals the total demand from the cities. The figure below shows the associated distribution network.

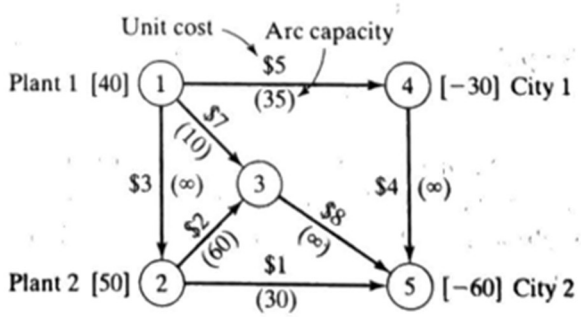

Fig1.: Maize Flour distribution network

Solution:

a. **Problem description:**

- **Supply Nodes:**
  - Plant 1 (Node 1): Supply = 40 thousand tons
  - Plant 2 (Node 2): Supply = 50 thousand tons
- **Demand Nodes:**
  - City 1 (Node 4): Demand = 30 thousand tons
  - City 2 (Node 5): Demand = 60 thousand tons
- **Network Balance:** Total Supply (40 + 50 = 90) equals Total Demand (30 + 60 = 90), so the network is balanced.

- **Arc Information:** Each arc has a unit cost (C_{ij}) and an arc capacity (U_{ij}). Lower bounds (L_{ij}) are assumed to be 0 unless specified.
- **Nodes:** 1, 2, 3, 4, 5
- **Arcs (cost, capacity):**
  - ➔ (1,4): $C_{14} = 5$, $U_{14} = 35$
  - ➔ (1,3): $C_{13} = 7$, $U_{13} = 10$
  - ➔ (1,2): $C_{12} = 3$, $U_{12} = \infty$
  - ➔ (2,3): $C_{23} = 2$, $U_{23} = 60$
  - ➔ (2,5): $C_{25} = 1$, $U_{25} = 30$
  - ➔ (3,4): $C_{34} = 4$, $U_{34} = \infty$
  - ➔ (3,5): $C_{35} = 8$, $U_{35} = \infty$
  - ➔ (4,5): $C_{45} = 4$, $U_{45} = \infty$

b. Iteration 0: Initial Basic Feasible Solution

Step 0: Determine a starting basic Feasible Solution

By inspection (Other methods stated previously could also be used), a basic feasible spanning tree is identified.

Basic Arcs (Solid Lines) are (1,3), (1,4), (2,3) and (3,5).

Nonbasic Arcs (Dashed Lines are (1,2), (2,5), (4,5), (3,4).

Initial flows (X_{ij}): $X_{13} = 10$, $X_{14} = 30$, $X_{23} = 50$, $X_{35} = 60$.

Let's verify flow conservation and capacities:

- ➔ **Node 1** (Supply 40): $X_{13} + X_{14} = 10 + 30 = 40$ (Satisfied)
  Arc (1,3) with Flow 10 ≤ Capacity 10 (At upper bound)
  Arc (1,4) with Flow 30 ≤ Capacity 35 (Within the capacity)
- ➔ **Node 2** (Supply 50): $X_{23} = 50$ (Satisfied)
  Arc (2,3) with Flow 50 ≤ Capacity 60 (Within capacity)
- ➔ **Node 3** (Transshipment): $X_{13} + X_{23} = 10 + 50 = 60$ and $X_{35} = 60$ (Satisfied as 60-60=0)
- ➔ **Node 4** (Demand 30): $X_{14} = 30$ (Satisfied)
- ➔ **Node 5** (Demand 60): $X_{35} = 60$ (Satisfied)

Thus, all conditions are met.

Diagram (Iteration 0):

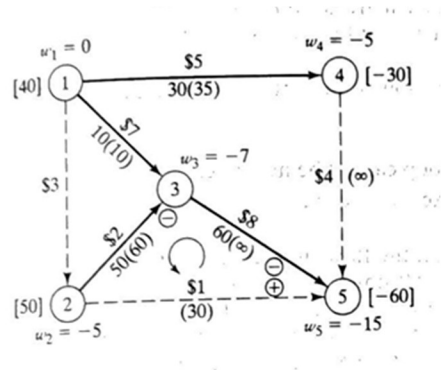

Fig2.: Iteration 0 | MCCFP illustrative example

Note: Arcs (1,2), (2,5) and (4,5) are nonbasic and their flows are 0.

c. **Iteration 1**

Step 1: Determine the Entering Arc (Optimality Condition)

1. **Compute Dual Variable (W_i):**
   Set $\boldsymbol{W_i = 0}$ then for,
   (1,3): $\boldsymbol{W_1 - W_3 = C_{13} \Longrightarrow\ 0 - W_3 = 7\ \Rightarrow W_3 = -7}$
   (1,4): $\boldsymbol{W_1 - W_4 = C_{14} \Longrightarrow\ 0 - W_4 = 5\ \Rightarrow W_4 = -5}$
   (2,3): $\boldsymbol{W_2 - W_3 = C_{23} \Longrightarrow\ W_2 - (-7) = 2\ \Rightarrow W_2 = -5}$
   (3,5): $\boldsymbol{W_3 - W_5 = C_{35} \Longrightarrow\ -7 - W_5 = 8\ \Rightarrow W_5 = -15}$
   So, the dual variables are: $\boldsymbol{W_1 = 0, W_2 = -5, W_3 = -7, W_4 = -5, W_5 = -15}$
2. **Compute Reduced Costs ($Z_{ij} - C_{ij}$) for Nonbasic Arcs:**
   (1,2): $\boldsymbol{Z_{12} - C_{12} = W_1 - W_2 - C_{12} = 0 - (-5) - 3 = 2}$
   (2,5): $\boldsymbol{Z_{25} - C_{25} = W_2 - W_5 - C_{25} = (-5) - (-15) - 1 = 9}$
   (4,5): $\boldsymbol{Z_{45} - C_{45} = W_4 - W_5 - C_{45} = (-5) - (-15) - 4 = 6}$
   (3,4): $\boldsymbol{Z_{34} - C_{34} = W_3 - W_4 - C_{34} = (-7) - (-5) - 4 = -6}$
3. **Select Entering Arc:**
   The most positive reduced cost for an arc at its lower bound is 9, corresponding to arc (2,5). Thus **arc (2,5) enters the basic solution.**

Step 2: Determine the Leaving Arc (Feasibility Condition) and Update the Solution

1. **Identify the Cycle:**
   Adding arc (2,5) to the current spanning tree creates a unique cycle: **2-5-3-2**
   Flow changes: $X_{25} \uparrow \Delta, X_{35} \downarrow \Delta, X_{23} \downarrow \Delta$
2. **Determine flow adjustment (Δ):**
   Current flows: $\boldsymbol{X_{13} = 10, X_{14} = 30, X_{23} = 50, X_{35} = 60}$
   Arcs where flow increases:
   (2,5): $\boldsymbol{X_{25} = 0,}$ Capacity $\boldsymbol{U_{25} = 30.}$ Max increase=30-0=30.
   Arcs where flow decreases:
   (3,5): $x_{35} = 60,$ lower bound $l_{35} = 0,$ Max decrease $= 60 - 0 = 60$

(2,3): $x_{23} = 50$, lower bound $l_{23} = 0$, Max decrease $= 50 - 0 = 50$

The maximum possible flow change Δ is the minimum of these values: min (30,60,50) = 30.

3. **Selecting Leaving arc:**
   Arc (2,5) is the first arc to hit its capacity limit (its own upper bound of 30) when its flow increases by $\Delta = 30$. In the context of the Network simplex, this means that while arc (2,5) is chosen to enter, its flow immediately becomes fixed at its upper bound. It effectively transitions from being a nonbasic variable at its lower bound to a **nonbasic variable at its upper bound**.
4. **Update Flows and Basis:**
   $x_{25} \leftarrow x_{25} + \Delta = 0 + 30 = 30$
   $x_{35} \leftarrow x_{35} - \Delta = 60 - 30 = 30$
   $x_{23} \leftarrow x_{23} - \Delta = 50 - 30 = 20$
   The new basic solution consists of arcs (1,3), (1,4), (2,3), and (3,5). Arc (2,5) is now nonbasic at its upper bound.
   Diagram (Iteration 1):

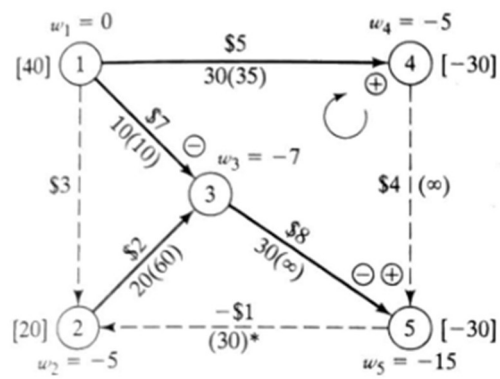

Fig3.: Iteration 1| MCCFP illustrative example

d. **Iteration 2**

Step 1: Determine the Entering Arc (Optimality Condition)

1. **Compute Dual Variables (W_i):**
   Set $\boldsymbol{W_1 = 0}$, then for
   (1,3): $w_1 - w_3 = c_{13} \Rightarrow 0 - w_3 = 7 \Rightarrow w_3 = -7$
   (1,4): $w_1 - w_4 = c_{14} \Rightarrow 0 - w_4 = 5 \Rightarrow w_4 = -5$
   (2,3): $w_2 - w_3 = c_{23} \Rightarrow w_2 - (-7) = 2 \Rightarrow w_2 = -5$
   (3,5): $w_3 - w_5 = c_{35} \Rightarrow -7 - w_5 = 8 \Rightarrow w_5 = -15$
   So, the dual variables are $\boldsymbol{W_1 = 0, W_2 = -5, W_3 = -7, W_4 = -5, W_5 = -15}$
2. **Compute Reduced Costs for Nonbasic Arcs:**
   (1,2)(at LB): $z_{12} - c_{12} = w_1 - w_2 - c_{12} = 0 - (-5) - 3 = 2$

$(2,5)(\text{at UB}): z_{25} - c_{25} = w_2 - w_5 - c_{25} = (-5) - (-15) - 1 = 9$. (At UB, a positive reduced cost means we cannot increase its flow. To decrease flow, we would need a negative reduced cost. So, this arc is not a candidate to enter by increasing flow).

$(4,5)(\text{at LB}): z_{45} - c_{45} = w_4 - w_5 - c_{45} = (-5) - (-15) - 4 = 6$

$(3,4)(\text{at LB}): z_{34} - c_{34} = w_3 - w_4 - c_{34} = (-7) - (-5) - 4 = -6$

3. **Selecting Entering Arc:**
   The most positive reduced cost for an arc at its lower bound is 6, corresponding to arc (4,5). Therefore, **arc (4,5) enters the basic solution.**

Step 2: Determine the Leaving Arc (Feasibility Condition) and Update the Solution:

1. **Identify the cycle:**
   Adding arc (4,5) to the current spanning tree creates a unique cycle: **4 - 1 - 3 - 5 - 4**.
   Flow changes: $x_{25} \uparrow \Delta, x_{35} \downarrow \Delta, x_{13} \downarrow \Delta, x_{14} \uparrow \Delta$
2. **Determine Flow Adjustment:** Current flows: $x_{13} = 10$, $x_{14} = 30$, $x_{23} = 20, x_{25} = 30, x_{35} = 30$
   Arcs where flow increases:
   $(4,5): x_{45} = 0, u_{45} = \infty$.Max increase $= \infty$.
   $(1,4): x_{14} = 30, u_{14} = 35$.Max increase $= 35 - 30 = 5$.
   Arcs where flow decreases:
   $(3,5): x_{35} = 30, \quad l_{35} = 0$.Max decrease $= 30 - 0 = 30$.
   $(1,3): x_{13} = 10, \quad l_{13} = 0$.Max decrease $= 10 - 0 = 10$.
   The maximum possible flow change is now min (∞, 5, 30, 10) =5.
3. **Select Leaving Arc:**
   Arc (1,4) hits its upper bound (35) when its flow increases by 5. Therefore, **Arc (1,4)** leaves the basic solution and becomes nonbasic at its upper bound.
4. **Update Flows and Basis:**
   $x_{45} \leftarrow x_{45} + \Delta = 0 + 5 = 5$
   $x_{35} \leftarrow x_{35} - \Delta = 30 - 5 = 25$
   $x_{13} \leftarrow x_{13} - \Delta = 10 - 5 = 5$
   $x_{14} \leftarrow x_{14} + \Delta = 30 + 5 = 35$ (Leaves basis, nonbasic at UB)
   The new basic solution consists of arcs (1,3), (2,3), (3,5), (4,5). Arc (1,4) is now nonbasic at its upper bound.

Diagram (Iteration 2):

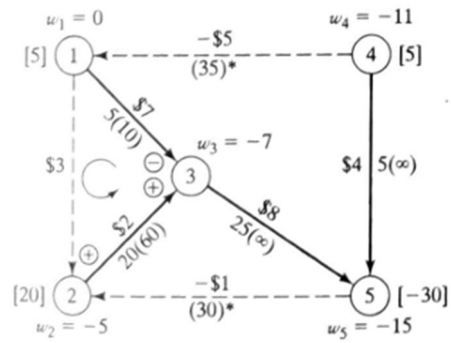

Fig4.: Iteration 2| MCCFP-illustrative example

e. **Iteration 3**

Step 1: Determine the Entering Arc (Optimality condition)

1. **Compute Dual Variables (W_i):**
   Set $\boldsymbol{W_1 = 0}$**,** then for
   $(1,3): w_1 - w_3 = c_{13} \Rightarrow 0 - w_3 = 7 \Rightarrow w_3 = -7$
   $(3,5): w_3 - w_5 = c_{35} \Rightarrow -7 - w_5 = 8 \Rightarrow w_5 = -15$
   $(4,5): w_4 - w_5 = c_{45} \Rightarrow w_4 - (-15) = 4 \Rightarrow w_4 = -11$
   $(2,3): w_2 - w_3 = c_{23} \Rightarrow w_2 - (-7) = 2 \Rightarrow w_2 = -5$
   So, the dual variables are $\boldsymbol{W_1 = 0, W_2 = -5, W_3 = -7, W_4 = -11, W_5 = -15}$
2. **Compute Reduced Costs for Nonbasic Arcs:**
   $(1,2)(\text{at LB}): \mathrm{z}_{12} - \mathrm{c}_{12} = \mathrm{w}_1 - \mathrm{w}_2 - \mathrm{c}_{12} = 0 - (-5) - 3 = 2$
   $(1,4)(\text{at UB}): \mathrm{z}_{14} - \mathrm{c}_{14} = \mathrm{w}_1 - \mathrm{w}_4 - \mathrm{c}_{14} = 0 - (-11) - 5 = 6$. (At UB, positive reduced cost means cannot increase flow. Not a candidate to enter by decreasing flow).
   $(2,5)(\text{at UB}): \mathrm{z}_{25} - \mathrm{c}_{25} = \mathrm{w}_2 - \mathrm{w}_5 - \mathrm{c}_{25} = (-5) - (-15) - 1 = 9$. (At UB, positive reduced cost means cannot increase flow. Not a candidate to enter by decreasing flow)
   $(3,4)(\text{at LB}): \mathrm{z}_{34} - \mathrm{c}_{34} = \mathrm{w}_3 - \mathrm{w}_4 - \mathrm{c}_{34} = (-7) - (-11) - 4 = 0$. (No improvement).
3. **Select entering arc:**
   The only nonbasic arc at its lower bound with a positive reduced cost is (1,2). Therefore, **arc (1,2) enters the basic solution.**

Step 2: Determine the Leaving Arc (Feasibility Condition) and Update the Solution.

1. **Identify the Cycle:**
   Adding arc (1,2) to the current spanning tree {(1,3), (2,3), (3,5), (4,5)}. Path from 1 to 2 using basic arcs: 1 → (1,3) → 3 ← (2,3) ← 2. So, the cycle is: **1 - 2 - 3 – 1.**
   Flow changes: $x_{12} \uparrow \Delta, x_{23} \uparrow \Delta, x_{13} \downarrow \Delta$

2. **Determine flow adjustment:**
   Current flows: $x_{13} = 5$, $x_{14} = 35$, $x_{23} = 20, x_{25} = 30, x_{35} = 25, x_{45} = 5$.
   Arcs where flow increases:
   $(1,2): x_{12} = 0, u_{12} = \infty$.Max increase $= \infty$.
   $(2,3): x_{23} = 20, u_{23} = 60$.Max increase $= 60 - 20 = 40$.
   Arcs where flow decreases:
   $(1,3): x_{13} = 5, \quad l_{13} = 0$.Max decrease $= 5 - 0 = 5$.
   The maximum possible flow change is min (∞,40, 5) =5.
3. **Select Leaving Arc:**
   Arc (1,3) hits its lower bound (0) when its flow decreases by Δ=5. Therefore, **arc (1,3) leaves the basic solution.**
4. **Update flows and Basis:**
   $x_{12} \leftarrow x_{12} + \Delta = 0 + 5 = 5$
   $x_{23} \leftarrow x_{23} + \Delta = 20 + 5 = 25$
   $x_{13} \leftarrow x_{13} - \Delta = 5 - 5 = 0$ (Leaves basis)
   The new basic solution consists of arcs (1,2), (2,3), (3,5), (4,5). Arc (1,3) is now nonbasic at its lower bound.

f. Iteration 4

Step 1: Determine the Entering Arc (Optimality condition).

1. **Compute Dual Variables:**
   Set $\boldsymbol{W_1 = 0}$**,** then for
   $(1,2): w_1 - w_2 = c_{12} \Rightarrow 0 - w_2 = 3 \Rightarrow w_2 = -3$
   $(2,3): w_2 - w_3 = c_{23} \Rightarrow -3 - w_3 = 2 \Rightarrow w_3 = -5$
   $(3,5): w_3 - w_5 = c_{35} \Rightarrow -5 - w_5 = 8 \Rightarrow w_5 = -13$
   $(4,5): w_4 - w_5 = c_{45} \Rightarrow w_4 - (-13) = 4 \Rightarrow w_4 = -9$
   So, the dual variables are $\boldsymbol{W_1 = 0, W_2 = -3, W_3 = -5, W_4 = -9, W_5 = -13}$
2. **Compute Reduced Costs for nonbasic Arcs:**
   $(1,3)(\text{at LB}): \mathrm{z}_{13} - \mathrm{c}_{13} = \mathrm{w}_1 - \mathrm{w}_3 - \mathrm{c}_{13} = 0 - (-5) - 7 = -2$
   $(1,4)(\text{at UB}): \mathrm{z}_{14} - \mathrm{c}_{14} = \mathrm{w}_1 - \mathrm{w}_4 - \mathrm{c}_{14} = 0 - (-9) - 5 = 4$. (At UB, positive reduced cost means cannot increase flow. Not a candidate.
   $(2,5)(\text{at UB}): \mathrm{z}_{25} - \mathrm{c}_{25} = \mathrm{w}_2 - \mathrm{w}_5 - \mathrm{c}_{25} = (-3) - (-13) - 1 = 9$. (At UB, positive reduced cost means cannot increase flow. Not a candidate.
   $(3,4)(\text{at LB}): \mathrm{z}_{34} - \mathrm{c}_{34} = \mathrm{w}_3 - \mathrm{w}_4 - \mathrm{c}_{34} = (-5) - (-9) - 4 = 0$ (No improvement).
3. **Select Entering Arc:**
   All nonbasic arcs at their lower bound have non-positive reduced costs. All nonbasic arcs at their upper bound have positively reduced costs (meaning we don't want to decrease their flow). Therefore, the current solution is **optimal.**

g. Optimal Solution

The optimal flows are:

$x_{12} = 5$ thousand tons
$x_{13} = 0$ thousand tons(nonbasic at lower bound)
$x_{14} = 35$ thousand tons (nonbasic at upper bound)
$x_{23} = 25$ thousand tons
$x_{25} = 30$ thousand tons (nonbasic at upper bound)
$x_{34} = 0$ thousand tons (nonbasic at lower bound)
$x_{35} = 25$ thousand tons
$x_{45} = 5$ thousand tons
Let's calculate the total cost with these flows:
$Z = (c_{12} \times x_{12}) + (c_{14} \times x_{14}) + (c_{23} \times x_{23}) + (c_{25} \times x_{25}) + (c_{35} \times x_{35}) + (c_{45} \times x_{45})$
$Z = (\$3 \times 5) + (\$5 \times 35) + (\$2 \times 25) + (\$1 \times 30) + (\$8 \times 25) + (\$4 \times 5)$
$Z = \$15 + \$175 + \$50 + \$30 + \$200 + \$20$
$Z = \$490$
Diagram (Optimal Solution):

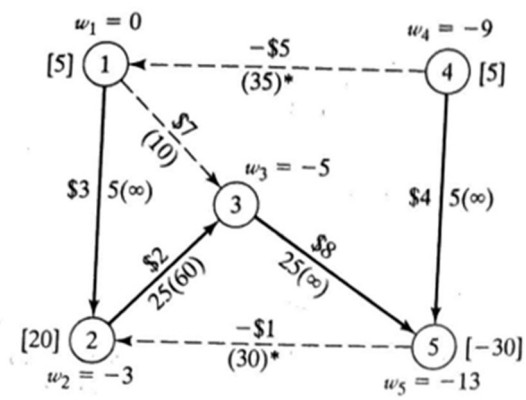

Fig5.: Optimal Solution| MCCFP illustrative example

Note: The diagram shows the final flows on the arcs. Arcs (1,3) and (3,4) are nonbasic at their lower bounds (flow 0). Arcs (1,4) and (2,5) are nonbasic at their upper bounds (flow 35 and 30, respectively).

4. RESULTS, PRESENTATION AND ANALYSIS

This analysis focuses on the benefits of applying MCCFP to maize distribution logistics in Eastern province and Kigali city, Rwanda.

4.1 Problem definition and Data Integration:

- **Supply districts (Nodes with positive net flow):**
  Nyagatare : 112,485.2434 Tons
  Kirehe : 48,783.3388 Tons

Gatsibo : 35,510.77112 Tons
Kayonza : 25,981.53448 Tons
Ngoma : 19,377.61984 Tons
Rwamagana : 5,020.96474 Tons
Bugesera : 1,596.93174 Tons

- **Demand districts**
  Gasabo : -23,525.0671 Tons
  Kicukiro : -14,983.11202 Tons
  Nyarugenge : -11,837.37898 Tons
- **EAX Warehouses and their capacities**
  Kayonza 1: 4,500 Tons
  Kayonza 2: 4,500 Tons
  Rwamagana-Musha: 1,500 Tons
  Bugesera-Mayange: 12,000 Tons
- **Arcs table:** shows the connections between supply districts, warehouses, and demand districts represented by arcs, each with defined distances, unit costs, truck capacities, and lower/upper bounds.

ID	From	To	Distance (km)	*Unit Cost (Rwfs/km/T)*	Total Cost (Rwfs/ T)	*Truck capacity (Tons/year)*	LiJ- Lower bound per trip (Tons)	LiJ- Lower bound (Tons/ year)	*Supply capacity (Tons/Year)*	*Demand (Tons/year)*	UIJ (upper bound)
1	Nyagatare	Kayonza 1	86.8	200	17360	4160	0.58	59.9	112,485	4,500	4,160
2	Nyagatare	Kayonza 2	106	200	21200	4160	0.47	49.1	112,485	4,500	4,160
3	Nyagatare	Rwamagana-Musha	110	200	22000	4160	0.45	47.3	112,485	1,500	1,500
4	Nyagatare	Bugesera-Mayange	173	200	34600	4160	0.29	30.1	112,485	12,000	4,160
5	Nyagatare	Gasabo	132	200	26400	4160	0.38	39.4	112,485	23,525	4,160
6	Nyagatare	Kicukiro	137	200	27400	4160	0.36	38.0	112,485	14,983	4,160
7	Nyagatare	Nyarugenge	134	200	26800	4160	0.37	38.8	112,485	11,837	4,160
8	Kirehe	Kayonza 1	61.4	200	12280	4160	0.81	84.7	48,783	4,500	4,160
9	Kirehe	Kayonza 2	62.4	200	12480	4160	0.80	83.3	48,783	4,500	4,160
10	Kirehe	Rwamagana-Musha	98.8	200	19760	4160	0.51	52.6	48,783	1,500	1,500
11	Kirehe	Bugesera-Mayange	104	200	20800	4160	0.48	50.0	48,783	12,000	4,160
12	Kirehe	Gasabo	145	200	29000	4160	0.34	35.9	48,783	23,525	4,160
13	Kirehe	Kicukiro	138	200	27600	4160	0.36	37.7	48,783	14,983	4,160
14	Kirehe	Nyarugenge	145	200	29000	4160	0.34	35.9	48,783	11,837	4,160
15	Rwamagana	Kayonza 1	43.7	200	8740	4160	1.14	119.0	5,021	4,500	4,160
16	Rwamagana	Kayonza 2	63.1	200	12620	4160	0.79	82.4	5,021	4,500	4,160
17	Rwamagana	Rwamagana-Musha	14.1	200	2820	4160	3.55	368.8	5,021	1,500	1,500
18	Rwamagana	Bugesera-Mayange	66.7	200	13340	4160	0.75	78.0	5,021	12,000	4,160
19	Rwamagana	Gasabo	38.3	200	7660	4160	1.31	135.8	5,021	23,525	4,160
20	Rwamagana	Kicukiro	31.5	200	6300	4160	1.59	165.1	5,021	14,983	4,160
21	Rwamagana	Nyarugenge	38	200	7600	4160	1.32	136.8	5,021	11,837	4,160
22	Kayonza	Kayonza 1	11.7	200	2340	4160	4.27	444.4	25,982	4,500	4,160
23	Kayonza	Kayonza 2	7.7	200	1540	4160	6.49	675.3	25,982	4,500	4,160
24	Kayonza	Rwamagana-Musha	49.1	200	9820	4160	1.02	105.9	25,982	1,500	1,500
25	Kayonza	Bugesera-Mayange	85.4	200	17080	4160	0.59	60.9	25,982	12,000	4,160
26	Kayonza	Gasabo	95.5	200	19100	4160	0.52	54.5	25,982	23,525	4,160
27	Kayonza	Kicukiro	88.8	200	17760	4160	0.56	58.6	25,982	14,983	4,160
28	Kayonza	Nyarugenge	95.2	200	19040	4160	0.53	54.6	25,982	11,837	4,160
29	Bugesera	Kayonza 1	98.9	200	19780	4160	0.51	52.6	1,597	4,500	1,597
30	Bugesera	Kayonza 2	103	200	20600	4160	0.49	50.5	1,597	4,500	1,597
31	Bugesera	Rwamagana-Musha	60.7	200	12140	4160	0.82	85.7	1,597	1,500	1,500
32	Bugesera	Bugesera-Mayange	11	200	2200	4160	4.55	472.7	1,597	12,000	1,597
33	Bugesera	Gasabo	33.2	200	6640	4160	1.51	156.6	1,597	23,525	1,597
34	Bugesera	Kicukiro	25.4	200	5080	4160	1.97	204.7	1,597	14,983	1,597
35	Bugesera	Nyarugenge	32.9	200	6580	4160	1.52	158.1	1,597	11,837	1,597
36	Ngoma	Kayonza 1	27.7	200	5540	4160	1.81	187.7	19,378	4,500	4,160
37	Ngoma	Kayonza 2	27.8	200	5560	4160	1.80	187.1	19,378	4,500	4,160
38	Ngoma	Rwamagana-Musha	65.1	200	13020	4160	0.77	79.9	19,378	1,500	1,500
39	Ngoma	Bugesera-Mayange	64.9	200	12980	4160	0.77	80.1	19,378	12,000	4,160
40	Ngoma	Gasabo	111	200	22200	4160	0.45	46.8	19,378	23,525	4,160
41	Ngoma	Kicukiro	105	200	21000	4160	0.48	49.5	19,378	14,983	4,160
42	Ngoma	Nyarugenge	111	200	22200	4160	0.45	46.8	19,378	11,837	4,160
43	Gatsibo	Kayonza 1	52.3	200	10460	4160	0.96	99.4	35,511	4,500	4,160
44	Gatsibo	Kayonza 2	71.6	200	14320	4160	0.70	72.6	35,511	4,500	4,160
45	Gatsibo	Rwamagana-Musha	75.8	200	15160	4160	0.66	68.6	35,511	1,500	1,500
46	Gatsibo	Bugesera-Mayange	145	200	29000	4160	0.34	35.9	35,511	12,000	4,160
47	Gatsibo	Gasabo	104	200	20800	4160	0.48	50.0	35,511	23,525	4,160
48	Gatsibo	Kicukiro	115	200	23000	4160	0.43	45.2	35,511	14,983	4,160
49	Gatsibo	Nyarugenge	107	200	21400	4160	0.47	48.6	35,511	11,837	4,160
50	Kayonza 1	Gasabo	83.8	200	16760	4160	0.60	62.1	4,500	23,525	4,160
51	Kayonza 2	Gasabo	103	200	20600	4160	0.49	50.5	4,500	23,525	4,160
52	Rwamagana-Mu	Gasabo	51.9	200	10380	4160	0.96	100.2	1,500	23,525	1,500
53	Bugesera-Mayan	Gasabo	44.1	200	8820	4160	1.13	117.9	12,000	23,525	4,160
54	Kayonza 1	Kicukiro	77.1	200	15420	4160	0.65	67.4	4,500	14,983	4,160
55	Kayonza 2	Kicukiro	96.5	200	19300	4160	0.52	53.9	4,500	14,983	4,160
56	Rwamagana-Mu	Kicukiro	45.2	200	9040	4160	1.11	115.0	1,500	14,983	1,500
57	Bugesera-Mayan	Kicukiro	36.4	200	7280	4160	1.37	142.9	12,000	14,983	4,160
58	Kayonza 1	Nyarugenge	83.4	200	16680	4160	0.60	62.4	4,500	11,837	4,160
59	Kayonza 2	Nyarugenge	103	200	20600	4160	0.49	50.5	4,500	11,837	4,160
60	Rwamagana-Mu	Nyarugenge	51.5	200	10300	4160	0.97	101.0	1,500	11,837	1,500
61	Bugesera-Mayan	Nyarugenge	43.7	200	8740	4160	1.14	119.0	12,000	11,837	4,160

Table1.: Dataset used in this project

- **Network illustration:**

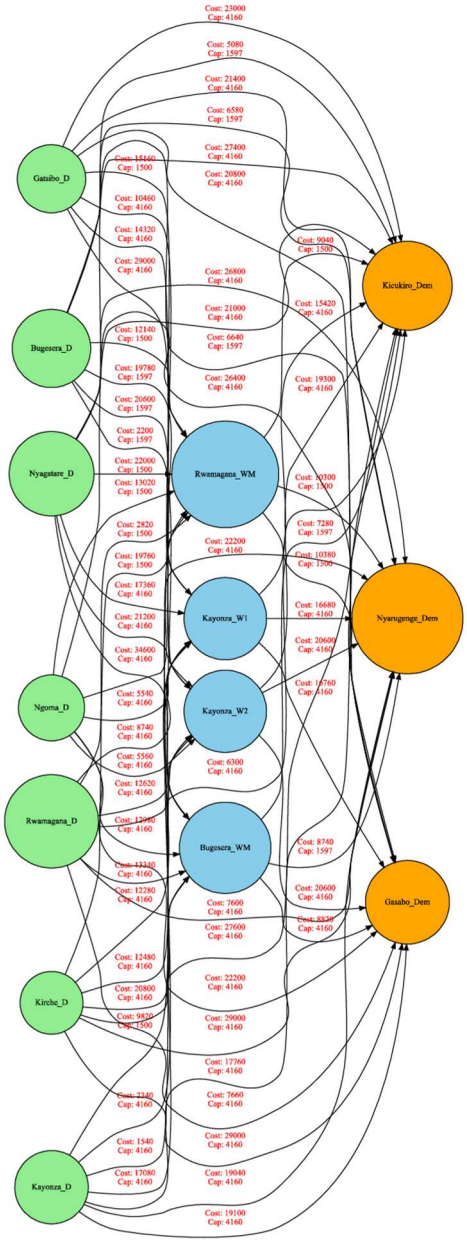

Fig6.: Initial arc connections for this project

Legend:

Green: Supply nodes (Eastern Province districts)

Blue: Transshipment nodes (EAX warehouses)

Orange: Demand nodes (Kigali city districts)

Edges: Transportation cost (Rwf/Ton), capacity (Tons/year)

4.2 Model implementation and solution

The MCCFP model was implemented using python's PuLp and SciPy optimization libraries. And the optimal solution is presented from the codes' output snip set pasted below:

```
Problem Status: Optimal

Optimal Total Transportation Cost: 933,252,049.40 Rwfs

Flows (Tons/Year):
  Kayonza_D -> Gasabo_Dem: 4,160.00 Tons (Cost: 19,100.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 54.50 Tons)
  Kayonza_D -> Kicukiro_Dem: 4,160.00 Tons (Cost: 17,760.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 58.60 Tons)
  Kayonza_D -> Kayonza_W2: 4,160.00 Tons (Cost: 1,540.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 675.30 Tons)
  Kayonza_D -> Nyarugenge_Dem: 4,160.00 Tons (Cost: 19,040.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 54.60 Tons)
  Kayonza_D -> Kayonza_W1: 4,160.00 Tons (Cost: 2,340.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 444.40 Tons)
  Ngoma_D -> Bugesera_WM: 4,160.00 Tons (Cost: 12,980.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 80.10 Tons)
  Ngoma_D -> Gasabo_Dem: 4,160.00 Tons (Cost: 22,200.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 46.80 Tons)
  Gatsibo_D -> Gasabo_Dem: 4,160.00 Tons (Cost: 20,800.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 50.00 Tons)
  Gatsibo_D -> Nyarugenge_Dem: 4,160.00 Tons (Cost: 21,400.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 48.60 Tons)
  Rwamagana_D -> Kicukiro_Dem: 4,160.00 Tons (Cost: 6,300.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 165.10 Tons)
  Kayonza_W2 -> Gasabo_Dem: 4,160.00 Tons (Cost: 20,600.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 50.50 Tons)
  Ngoma_D -> Nyarugenge_Dem: 3,517.38 Tons (Cost: 22,200.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 46.80 Tons)
  Bugesera_WM -> Gasabo_Dem: 2,563.00 Tons (Cost: 8,820.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 117.90 Tons)
  Kayonza_W1 -> Kicukiro_Dem: 2,538.90 Tons (Cost: 15,420.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 67.40 Tons)
  Kayonza_W1 -> Gasabo_Dem: 1,961.10 Tons (Cost: 16,760.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 62.10 Tons)
  Bugesera_WM -> Kicukiro_Dem: 1,597.00 Tons (Cost: 7,280.00 Rwfs/T, UIJ: 1,597.00 Tons, LIJ: 142.90 Tons)
  Bugesera_D -> Kicukiro_Dem: 1,596.93 Tons (Cost: 5,080.00 Rwfs/T, UIJ: 1,597.00 Tons, LIJ: 204.70 Tons)
  Rwamagana_WM -> Gasabo_Dem: 1,500.00 Tons (Cost: 10,380.00 Rwfs/T, UIJ: 1,500.00 Tons, LIJ: 100.20 Tons)
  Kayonza_D -> Rwamagana_WM: 1,500.00 Tons (Cost: 9,820.00 Rwfs/T, UIJ: 1,500.00 Tons, LIJ: 105.90 Tons)
  Ngoma_D -> Kicukiro_Dem: 930.28 Tons (Cost: 21,000.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 49.50 Tons)
  Rwamagana_D -> Gasabo_Dem: 860.96 Tons (Cost: 7,660.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 135.80 Tons)
  Ngoma_D -> Kayonza_W1: 340.00 Tons (Cost: 5,540.00 Rwfs/T, UIJ: 4,160.00 Tons, LIJ: 187.70 Tons)

Supply Utilization:
  Nyagatare_D: 0.00 / 112,485.24 Tons supplied
  Kirehe_D: 0.00 / 48,783.34 Tons supplied
  Rwamagana_D: 5,020.96 / 5,020.96 Tons supplied
  Kayonza_D: 22,300.00 / 25,981.53 Tons supplied
  Bugesera_D: 1,596.93 / 1,596.93 Tons supplied
  Ngoma_D: 13,107.66 / 19,377.62 Tons supplied
  Gatsibo_D: 8,320.00 / 35,510.77 Tons supplied

Demand Fulfillment:
  Gasabo_Dem: 23,525.07 / 23,525.07 Tons demanded
  Kicukiro_Dem: 14,983.11 / 14,983.11 Tons demanded
  Nyarugenge_Dem: 11,837.38 / 11,837.38 Tons demanded
```

Fig7.: Python MCCFP solver for this project

4.3 Key insights

- The total supply greatly exceeds the total demand, indicating that the system has enough capacity to meet maize needs for the target regions. The challenge then is not availability, but efficient allocation and routing.
- The model minimized the **total annual transport cost** across all active routes to **933,252,049.40 RWF** (≈ $750,000). This value represents the lowest possible cost to meet current Kigali city maize demand.

- 100% of the demand is fulfilled (Proven by the Demand fulfillment of the code output above). This proves the model's effectiveness in minimizing the total maize distribution logistics cost.
- As depicted above in the code output snip set, the optimizer selected 22 distinct arcs (out of 61 possible), focusing on the **lowest-cost** combinations that respect capacity and feasibility.
- Nyagatare and Kirehe supplies remained unused, likely due to higher transport costs to the destination districts.
- Over one-third of maize moved through EAX warehouses, proving their strategic value as intermediaries

4.4 Visualizations

1. **Optimal network**

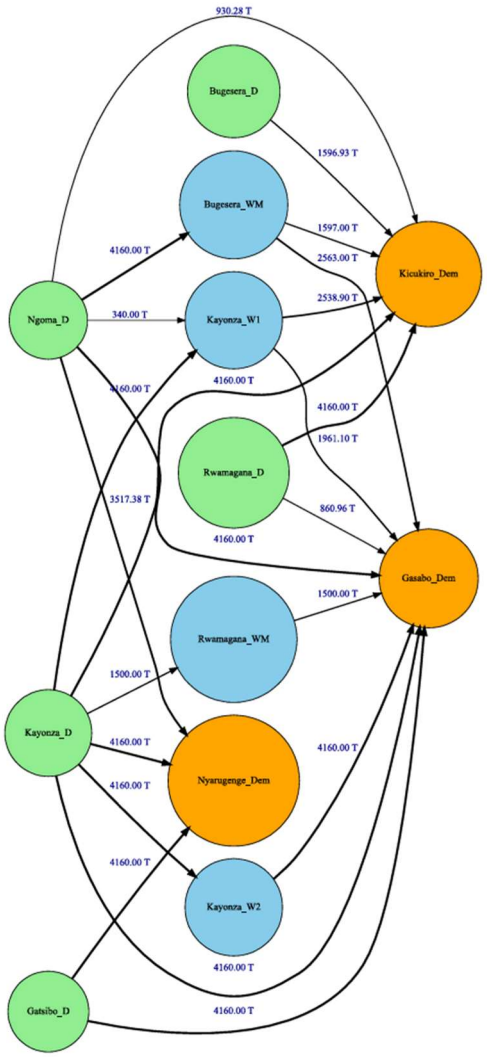

Fig8.: Optimal solution arc connections

Legend:
Green: Supply nodes (Eastern Province districts)
Blue: Transshipment nodes (EAX Warehouses)
Orange: Demand nodes (Kigali city districts)
Edges: Flow in Tons/year.

2. **Supply utilization per district**

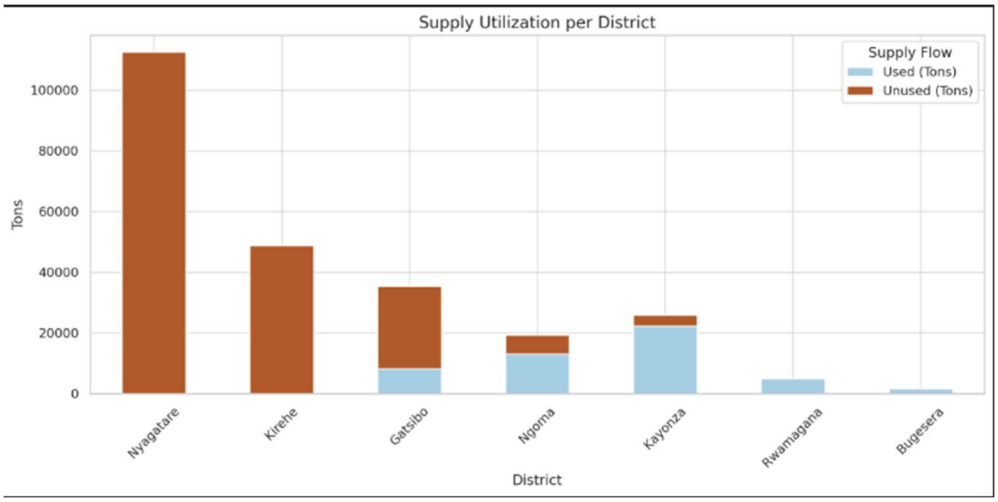

Fig9.: Supply utilization per district

Nyagatare and Kirehe, despite being top producers, had **0% utilization**, while **Rwamagana and Bugesera** were fully used. This reflects how **distance and cost** influenced the model's decisions to arrive at a minimum distribution strategy.

3. **Demand Fulfillment per District**

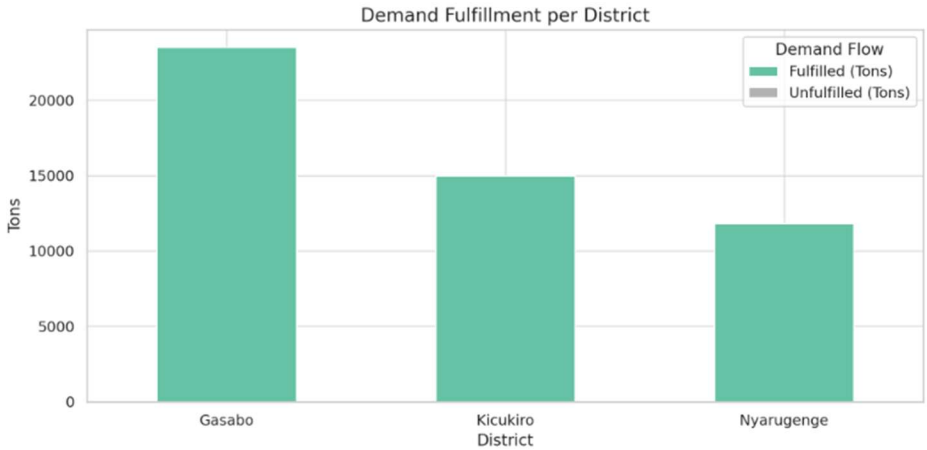

Fig10.: Demand Fulfillment per District

This chart confirms that **100% of maize demand** was fulfilled for all three districts: **Gasabo, Kicukiro, and Nyarugenge**. There are no unfulfilled bars, indicating a fully successful distribution strategy.

4. **EAX warehouse utilization**

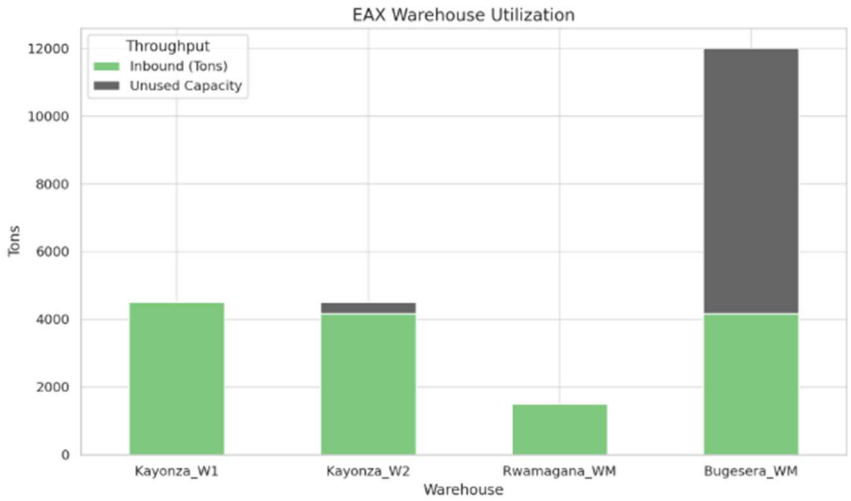

Fig11.: EAX warehouse utilization

Even though Bugesera_WM could have absorbed more volume, the **MCCFP model prioritized cost-efficiency over equal utilization**, favoring **warehouses with better geographic cost leverage**. This reveals a critical insight: **warehouse location matters as much as its size**, and future logistics planning should weigh **both capacity and transport economics** in infrastructure development.

5. DISCUSSION, CONCLUSION & Recommendations

5.1 Discussion

This study set out to explore how the **Minimum Cost Capacitated Flow Problem (MCCFP)** could optimize maize distribution from Rwanda's maize-surplus districts in Eastern province to high-demand urban districts of Kigali city, leveraging the **East Africa Exchange (EAX) warehouse infrastructure**.

The results revealed a **clear optimization advantage**: the model was able to fully meet maize demand at **minimal transport cost**, while respecting constraints like truck capacity and warehouse limits. However, it also uncovered **important operational realities**:

- **Cost-efficient routing dominates decisions**: major supply districts like Nyagatare and Kirehe were excluded due to higher distance-related costs.

- **Warehouses mattered**, but not equally. Despite its large capacity, Bugesera_Mayange warehouse was underused because routes through it were generally more expensive, showing how location and cost per kilometer can outweigh storage size.

  This brings to light a key insight into logistics planning: physical availability does not always translate into optimal utility unless transport efficiency is also considered.

5.2 Conclusion

The MCCFP model proved to be a **powerful decision-support tool** for planning maize logistics in Rwanda. It successfully:

- Fulfilled 100% of the maize demand in Kigali districts.
- Minimized overall transport costs to **~933 million RWF**.
- Optimized the use of existing infrastructure such as EAX warehouses and 20-ton trucks.

The model not only improved cost-effectiveness but also highlighted logistical blind spots-such as over-reliance on certain hubs, and underutilization of high-capacity producers due to distance penalties. This insight can help policymakers and supply chain managers realign storage, pricing, and collection strategies for future planning.

5.3 Recommendations

Based on the findings of this study, the following practical recommendations are suggested:

1. **Improve Data Collection**
   More accurate data on fuel prices, road conditions, truck capacities, and travel times would improve the model's accuracy and real-world usefulness.

2. **Apply the Model to Other Cases**
   The MCCFP approach can be applied to the distribution of other crops or goods and extended to more regions in Rwanda.

3. **Guide Warehouse Placement**
   Future warehouse locations should be planned to use cost-efficient models that consider transport distances and regional demand.

4. **Refine Trucking Assumptions**
   Future models should reflect real-life variations in truck size, loading time, and number of trips per year to improve planning.

5. **Consider a Broader Scope**
   This study looked at inter-district movement only. Expanding the model to cover local (intra-district) or national-level logistics could offer even deeper insights.

6. References

1. Balcik, B., & Beamon, B. M. (2008). *Facility location in humanitarian relief*. *International Journal of Logistics*, 11(2), 101–121. https://doi.org/10.1080/13675560701561789
2. Chen, X., He, L., & Zhang, Y. (2020). *Optimizing vaccine distribution using capacitated flow models*. *Operations Research for Health Care*, 25, 100255. https://doi.org/10.1016/j.orhc.2020.100255
3. Cox, A., Mugisha, J., & Niyitanga, J. (2020). *Structured grain trade in Rwanda: Benefits and barriers*. *Rwanda Journal of Agricultural Sciences*, 3(1), 44–59.
4. FAO. (2023). *The State of Food and Agriculture 2023*. Food and Agriculture Organization of the United Nations.
5. Grote, U., Fasse, A., Nguyen, T. T., & Erenstein, O. (2021). *Food loss and waste in developing countries: A review*. *Sustainability*, 13(10), 5443.
6. Klibi, W., & Martel, A. (2012). *Modeling approaches for the design of resilient supply chains*. *International Journal of Production Economics*, 135(2), 882–897.
7. MINAGRI. (2022). *Agricultural Season Report: Crop Production Statistics*. Ministry of Agriculture and Animal Resources, Rwanda.
8. Santpoort, R. (2020). *The drivers of maize price volatility in sub-Saharan Africa*. *Food Policy*, 91, 101824.
9. USAID. (2019). *East Africa regional maize market assessment*. U.S. Agency for International Development.
10. Taha, H. A. (2003). *Operations research: An introduction* (7th ed.). Prentice Hall of India.
11. Winston, W. L. (2004). *Operations research: Applications and algorithms* (4th ed.). Duxbury Press.
12. OpenAI. (2024). *ChatGPT* (June 2024 version) [Large language model]. https://chat.openai.com
13. Google. (2024). *Gemini* [Conversational AI model]. https://gemini.google.com
14. Google Research. (2024). *Google Colaboratory (Colab)* [Cloud-based Python notebook platform]. https://colab.research.google.com
15. National Institute of Statistics of Rwanda (NISR). (2024). *Seasonal Agricultural Survey 2024*. Kigali, Rwanda: NISR.
16. National Institute of Statistics of Rwanda (NISR). (2021). *Rwanda Food Balance Sheet 2017–2021*. Kigali, Rwanda: NISR.
17. National Institute of Statistics of Rwanda (NISR). (2022). *5th Rwanda Population and Housing Census – Provisional Results*. Kigali, Rwanda: NISR.

7. Appendix

Python solver codes:

```
# --- 1. Install the CBC Solver (Essential for Google Colab/Ubuntu-based environments) ---
# This ensures that PuLP can find and use the necessary solver.
!sudo apt-get install coinor-cbc
# --- 2. Import Necessary Libraries ---
import pulp
import pandas as pd
from io import StringIO
# --- 3. Configuration and Debugging Flags ---
ENABLE_LIJ_CONSTRAINTS = False
# --- 4. Raw Data Table (UPDATED with corrected warehouse-to-demand connections) ---
raw_data = """ID	From	To	"Distance (km)""Unit Cost (Rwfs/km/T)"	"Total Cost
(Rwfs/ T)"	"Truck capacity (Tons/year)"	"LiJ- Lower bound per trip (Tons)"	"LiJ- Lower
bound (Tons/ year)"	"Supply capacity (Tons/Year)"	"Demand (Tons/year)"	UIJ (upper
bound)
1	Nyagatare	Kayonza 1	86.8	200	17360	4160	0.58	59.9
	112485.2434	4500	4160
2	Nyagatare	Kayonza 2	106	200	21200	4160	0.47	49.1
	112485.2434	4500	4160
3	Nyagatare	Rwamagana-Musha	110	200	22000	4160	0.45	47.3
	112485.2434	1500	1500
4	Nyagatare	Bugesera-Mayange	173	200	34600	4160	0.29	30.1
	112485.2434	12000	4160
5	Nyagatare	Gasabo 132	200	26400	4160	0.38	39.4	112485.2434
	23525.0671	4160
6	Nyagatare	Kicukiro	137	200	27400	4160	0.36	38.0
	112485.2434	14983.11202	4160
7	Nyagatare	Nyarugenge	134	200	26800	4160	0.37	38.8
	112485.2434	11837.37898	4160
8	Kirehe	Kayonza 1	61.4	200	12280	4160	0.81	84.7	48783.3388
	4500	4160
9	Kirehe	Kayonza 2	62.4	200	12480	4160	0.80	83.3	48783.3388
	4500	4160
10	Kirehe	Rwamagana-Musha	98.8	200	19760	4160	0.51	52.6
	48783.3388	1500	1500
11	Kirehe	Bugesera-Mayange	104	200	20800	4160	0.48	50.0
	48783.3388	12000	4160
12	Kirehe	Gasabo 145	200	29000	4160	0.34	35.9	48783.3388
	23525.0671	4160
13	Kirehe	Kicukiro	138	200	27600	4160	0.36	37.7	48783.3388
	14983.11202	4160
14	Kirehe	Nyarugenge	145	200	29000	4160	0.34	35.9	48783.3388
	11837.37898	4160
```

15	Rwamagana	Kayonza 1	43.7	200	8740	4160	1.14	119.0	5020.96474	4500	4160	
16	Rwamagana	Kayonza 2	63.1	200	12620	4160	0.79	82.4	5020.96474	4500	4160	
17	Rwamagana	Rwamagana-Musha	14.1	200	2820	4160	3.55	368.8	5020.96474	1500	1500	
18	Rwamagana	Bugesera-Mayange	66.7	200	13340	4160	0.75	78.0	5020.96474	12000	4160	
19	Rwamagana	Gasabo	38.3	200	7660	4160	1.31	135.8	5020.96474	23525.0671	4160	
20	Rwamagana	Kicukiro	31.5	200	6300	4160	1.59	165.1	5020.96474	14983.11202	4160	
21	Rwamagana	Nyarugenge	38	200	7600	4160	1.32	136.8	5020.96474	11837.37898	4160	
22	Kayonza	Kayonza 1	11.7	200	2340	4160	4.27	444.4	25981.53448	4500	4160	
23	Kayonza	Kayonza 2	7.7	200	1540	4160	6.49	675.3	25981.53448	4500	4160	
24	Kayonza	Rwamagana-Musha	49.1	200	9820	4160	1.02	105.9	25981.53448	1500	1500	
25	Kayonza	Bugesera-Mayange	85.4	200	17080	4160	0.59	60.9	25981.53448	12000	4160	
26	Kayonza	Gasabo	95.5	200	19100	4160	0.52	54.5	25981.53448	23525.0671	4160	
27	Kayonza	Kicukiro	88.8	200	17760	4160	0.56	58.6	25981.53448	14983.11202	4160	
28	Kayonza	Nyarugenge	95.2	200	19040	4160	0.53	54.6	25981.53448	11837.37898	4160	
29	Bugesera	Kayonza 1	98.9	200	19780	4160	0.51	52.6	1596.93174	4500	1597	
30	Bugesera	Kayonza 2	103	200	20600	4160	0.49	50.5	1596.93174	4500	1597	
31	Bugesera	Rwamagana-Musha	60.7	200	12140	4160	0.82	85.7	1596.93174	1500	1500	
32	Bugesera	Bugesera-Mayange	11	200	2200	4160	4.55	472.7	1596.93174	12000	1597	
33	Bugesera	Gasabo	33.2	200	6640	4160	1.51	156.6	1596.93174	23525.0671	1597	
34	Bugesera	Kicukiro	25.4	200	5080	4160	1.97	204.7	1596.93174	14983.11202	1597	
35	Bugesera	Nyarugenge	32.9	200	6580	4160	1.52	158.1	1596.93174	11837.37898	1597	
36	Ngoma	Kayonza 1	27.7	200	5540	4160	1.81	187.7	19377.61984	4500	4160	
37	Ngoma	Kayonza 2	27.8	200	5560	4160	1.80	187.1	19377.61984	4500	4160	
38	Ngoma	Rwamagana-Musha	65.1	200	13020	4160	0.77	79.9	19377.61984	1500	1500	
39	Ngoma	Bugesera-Mayange	64.9	200	12980	4160	0.77	80.1	19377.61984	12000	4160	

```
40	Ngoma Gasabo 111	200	22200	4160	0.45	46.8		19377.61984
	23525.0671	4160
41	Ngoma Kicukiro	105	200	21000	4160	0.48	49.5		19377.61984
	14983.11202	4160
42	Ngoma Nyarugenge	111	200	22200	4160	0.45	46.8		19377.61984
	11837.37898	4160
43	Gatsibo Kayonza 1	52.3	200	10460	4160	0.96	99.4		35510.77112
	4500	4160
44	Gatsibo Kayonza 2	71.6	200	14320	4160	0.70	72.6		35510.77112
	4500	4160
45	Gatsibo Rwamagana-Musha	75.8	200	15160	4160	0.66	68.6
	35510.77112	1500	1500
46	Gatsibo Bugesera-Mayange	145	200	29000	4160	0.34	35.9
	35510.77112	12000	4160
47	Gatsibo Gasabo 104	200	20800	4160	0.48	50.0		35510.77112
	23525.0671	4160
48	Gatsibo Kicukiro	115	200	23000	4160	0.43	45.2		35510.77112
	14983.11202	4160
49	Gatsibo Nyarugenge	107	200	21400	4160	0.47	48.6		35510.77112
	11837.37898	4160
50	Kayonza 1	Gasabo 83.8	200	16760	4160	0.60	62.1		4500
	23525.0671	4160
51	Kayonza 2	Gasabo 103	200	20600	4160	0.49	50.5		4500
	23525.0671	4160
52	Rwamagana-Musha	Gasabo 51.9	200	10380	4160	0.96	100.2		1500
	23525.0671	1500
53	Bugesera-Mayange	Gasabo 44.1	200	8820	4160	1.13	117.9
	12000	23525.0671	4160
54	Kayonza 1	Kicukiro	77.1	200	15420	4160	0.65	67.4		4500
	14983.11202	4160
55	Kayonza 2	Kicukiro	96.5	200	19300	4160	0.52	53.9		4500
	14983.11202	4160
56	Rwamagana-Musha	Kicukiro	45.2	200	9040	4160	1.11	115.0
	1500	14983.11202	1500
57	Bugesera-Mayange	Kicukiro	36.4	200	7280	4160	1.37	142.9
	12000	14983.11202	1597
58	Kayonza 1	Nyarugenge	83.4	200	16680	4160	0.60	62.4		4500
	11837.37898	4160
59	Kayonza 2	Nyarugenge	103	200	20600	4160	0.49	50.5		4500
	11837.37898	4160
60	Rwamagana-Musha	Nyarugenge	51.5	200	10300	4160	0.97	101.0
	1500	11837.37898	1500
61	Bugesera-Mayange	Nyarugenge	43.7	200	8740	4160	1.14	119.0
	12000	11837.37898	1597
"""
# Read the data into a pandas DataFrame for easier processing
df = pd.read_csv(StringIO(raw_data), sep='\t')
# Clean column names (remove quotes and extra spaces)
df.columns = df.columns.str.replace('"', '').str.strip()
# --- 5. Node Name Mapping for Consistency ---
# Standardize names to avoid typos and ensure unique identification in the model.
```

```
node_name_map = {
    'Nyagatare': 'Nyagatare_D',       # District/Supply Node
    'Kirehe': 'Kirehe_D',             # District/Supply Node
    'Rwamagana': 'Rwamagana_D',       # District/Supply Node
    'Kayonza': 'Kayonza_D',           # District/Supply Node
    'Bugesera': 'Bugesera_D',         # District/Supply Node
    'Ngoma': 'Ngoma_D',               # District/Supply Node
    'Gatsibo': 'Gatsibo_D',           # District/Supply Node
    'Kayonza 1': 'Kayonza_W1',        # Warehouse 1
    'Kayonza 2': 'Kayonza_W2',        # Warehouse 2
    'Rwamagana-Musha': 'Rwamagana_WM',# Rwamagana Warehouse
    'Bugesera-Mayange': 'Bugesera_WM',# Bugesera Warehouse
    'Gasabo': 'Gasabo_Dem',           # Demand Destination
    'Kicukiro': 'Kicukiro_Dem',       # Demand Destination
    'Nyarugenge': 'Nyarugenge_Dem'    # Demand Destination
}
# Apply the mapping to 'From' and 'To' columns in the DataFrame
df['From'] = df['From'].replace(node_name_map)
df['To'] = df['To'].replace(node_name_map)
# --- 6. Data Extraction for Model Parameters ---
# Initialize dictionaries to hold unique node properties
supply_nodes = {}         # {supply_node_name: capacity}
demand_nodes = {}         # {demand_node_name: requirement}
eax_warehouse_capacities = {} # {warehouse_node_name: throughput_capacity}
# Arc-specific data
costs = {}                # {from_node: {to_node: cost}}
arc_capacities = {}       # {from_node: {to_node: UIJ_upper_bound}}
arc_lower_bounds = {}     # {from_node: {to_node: LiJ_lower_bound}}
all_nodes = set()         # Collects all unique nodes in the network
# Populate data structures by iterating through the DataFrame
for index, row in df.iterrows():
    f = row['From']
    t = row['To']
    # Populate supply_nodes (only for nodes ending with '_D' and appearing as 'From')
    if f.endswith('_D') and f not in supply_nodes:
        supply_nodes[f] = row['Supply capacity (Tons/Year)']
    # Populate demand_nodes (only for nodes ending with '_Dem' and appearing as 'To')
    if t.endswith('_Dem') and t not in demand_nodes:
        demand_nodes[t] = row['Demand (Tons/year)']
    # Populate eax_warehouse_capacities (for specific warehouse names appearing as 'To')
    if (t.startswith('Kayonza_W') or t.endswith('_WM')) and t not in eax_warehouse_capacities:
        eax_warehouse_capacities[t] = row['Demand (Tons/year)']
    # Populate costs, arc_capacities (UIJ), and arc_lower_bounds (LiJ)
    if f not in costs:
        costs[f] = {}
    costs[f][t] = row['Total Cost (Rwfs/ T)']
    if f not in arc_capacities:
        arc_capacities[f] = {}
    arc_capacities[f][t] = row['UIJ (upper bound)']
    if f not in arc_lower_bounds:
        arc_lower_bounds[f] = {}
```

```
    arc_lower_bounds[f][t] = row['LiJ- Lower bound (Tons/ year)']
    # Add both 'From' and 'To' nodes to the set of all nodes in the network
    all_nodes.add(f)
    all_nodes.add(t)
# --- 7. Identify Node Categories for Model Formulation ---
supply_districts = list(supply_nodes.keys())
demand_districts = list(demand_nodes.keys())
warehouses = list(eax_warehouse_capacities.keys())
# Transshipment nodes are any nodes that are not purely a supply district or a demand district.
# They act as intermediate points where flow can arrive and then depart.
transshipment_nodes = [node for node in all_nodes if node not in supply_districts and node not in
demand_districts]
# --- 8. Initial Feasibility Check: Total Supply vs. Total Demand ---
# This is crucial for diagnosing infeasibility before running the solver.
total_supply_available = sum(supply_nodes.values())
total_demand_required = sum(demand_nodes.values())
print(f"--- Initial Feasibility Check ---")
print(f"Total Supply Available: {total_supply_available:,.2f} Tons")
print(f"Total Demand Required: {total_demand_required:,.2f} Tons")
if total_supply_available < total_demand_required:
    print("\nWARNING: Total supply is LESS than total demand. The problem is inherently
INFEASIBLE.")
    print("        You cannot meet all demand with the current supply levels.")
elif total_supply_available == total_demand_required:
    print("\nTotal supply EQUALS total demand. The problem is balanced, but may still be infeasible
due to routing/capacities.")
else:
    print("\nTotal supply is GREATER than total demand. The problem should be able to meet demand,
assuming sufficient capacities.")
print("---------------------------------\n")
# --- 9. Create the LP Problem ---
# Define the optimization problem: Minimize total transportation cost.
prob = pulp.LpProblem("MaizeDistributionNetwork", pulp.LpMinimize)
# --- 10. Define Decision Variables ---
# 'flow[(f, t)]' represents the amount of maize (in Tons/Year) transported from node 'f' to node 't'.
flow = pulp.LpVariable.dicts("Flow",
                             ((f, t) for f in all_nodes for t in all_nodes if f in costs and t in costs[f]),
                             lowBound=0,  # Flow cannot be negative
                             cat='Continuous') # Flow can be any non-negative real number
# --- 11. Objective Function ---
# Minimize the total transportation cost, which is the sum of (flow * cost) for all arcs.
prob += pulp.lpSum(flow[f, t] * costs[f][t] for f, t in flow), "Total_Transportation_Cost"
# --- 12. Add Constraints to the Problem ---
# a. Supply Constraints: Total outflow from each supply district cannot exceed its supply capacity.
for s in supply_districts:
    # Ensure the sum only considers arcs that actually exist in the 'flow' variables
    outgoing_arcs = [(f_key, t) for f_key, t in flow if f_key == s]
    if outgoing_arcs: # Only add constraint if there are outgoing arcs
        prob += pulp.lpSum(flow[s, t] for s, t in outgoing_arcs) <= supply_nodes.get(s, 0),
f"Supply_Limit_{s}"
    else:
```

```
        print(f"Warning: Supply district {s} has no outgoing arcs defined in the data. Its supply cannot be
utilized.")
# b. Demand Constraints: Total inflow to each demand district must meet its demand requirement.
for d in demand_districts:
    # Ensure the sum only considers arcs that actually exist in the 'flow' variables
    incoming_arcs = [(f, t_key) for f, t_key in flow if t_key == d]
    if incoming_arcs: # Only add constraint if there are incoming arcs
        prob += pulp.lpSum(flow[f, d] for f, d in incoming_arcs) >= demand_nodes.get(d, 0),
f"Demand_Fulfillment_{d}"
    else:
        print(f"Warning: Demand district {d} has no incoming arcs defined in the data. Its demand cannot
be met.")
# c. Flow Conservation at Transshipment Nodes: Inflow must equal outflow for intermediate nodes.
for node in transshipment_nodes:
    incoming_flow_sum = pulp.lpSum(flow[f, node] for f, t in flow if t == node)
    outgoing_flow_sum = pulp.lpSum(flow[node, t] for f, t in flow if f == node)
    prob += incoming_flow_sum == outgoing_flow_sum, f"Flow_Conservation_{node}"
# d. Arc Capacity Constraints (UIJ - Upper Bound for each link):
#    The flow on any arc cannot exceed its specified upper capacity.
for f, t in flow:
    if f in arc_capacities and t in arc_capacities[f]:
        prob += flow[f, t] <= arc_capacities[f][t], f"Arc_Capacity_UIJ_{f}_to_{t}"
    else:
        # Fallback for data inconsistencies: assign a very high capacity if not found.
        # This warning helps identify missing data points.
        print(f"Warning: UIJ Arc capacity not found for {f} to {t}. Setting a very high upper bound.")
        prob += flow[f, t] <= 10**9
# e. Arc Lower Bound Constraints (LiJ - Lower Bound for each link):
#    The flow on any arc must be at least its specified lower bound (if ENABLE_LIJ_CONSTRAINTS
is True).
if ENABLE_LIJ_CONSTRAINTS:
    for f, t in flow:
        if f in arc_lower_bounds and t in arc_lower_bounds[f]:
            lij_val = arc_lower_bounds[f][t]
            if lij_val > 0: # Only add constraint if the lower bound is meaningful (greater than zero)
                prob += flow[f, t] >= lij_val, f"Arc_Capacity_LIJ_{f}_to_{t}"
        # If LiJ is not found or is 0, the variable's default lowBound=0 applies, which is correct.
# f. Nodal Capacity Constraints for EAX Warehouses (Throughput Capacity):
#    The total flow entering OR leaving a warehouse cannot exceed its capacity.
#    In a transshipment model, usually the *sum* of inflow or outflow (not both) is limited by capacity.
for w in warehouses:
    total_inflow_to_warehouse = pulp.lpSum(flow[f, w] for f, t_key in flow if t_key == w)
    total_outflow_from_warehouse = pulp.lpSum(flow[w, t] for f_key, t in flow if f_key == w)
    # Limiting total inflow to capacity
    prob += total_inflow_to_warehouse <= eax_warehouse_capacities.get(w, 0),
f"Warehouse_Inflow_Limit_{w}"
    # Limiting total outflow to capacity (typically inflow or outflow is sufficient, but both ensures
robustness)
    prob += total_outflow_from_warehouse <= eax_warehouse_capacities.get(w, 0),
f"Warehouse_Outflow_Limit_{w}"
# --- 13. Solve the Problem ---
```

```
# Use the COIN_CMD solver, which is robust for continuous linear problems.
# msg=True provides verbose output from the solver, which can be useful for debugging.
print("\n--- Running the Optimization Solver ---")
prob.solve(pulp.COIN_CMD(msg=True))
print("--- Solver Finished ---\n")
# --- 14. Print Results ---
print(f"Problem Status: {pulp.LpStatus[prob.status]}\n")
# Use integer values for comparison to avoid potential AttributeError issues with pulp.LpStatus
if prob.status == 1: # Represents pulp.LpStatus.Optimal
    print(f"Optimal Total Transportation Cost: {pulp.value(prob.objective):,.2f} Rwfs\n")
    print("Flows (Tons/Year):")
    # Sort flows for readability and filter out very small (effectively zero) flows
    sorted_flows = sorted([((f, t), flow[f, t].varValue) for f, t in flow if flow[f, t].varValue is not None
and flow[f, t].varValue > 0.001],
                          key=lambda item: item[1], reverse=True)
    for (f, t), val in sorted_flows:
        # Using .get() for robustness in case a cost/capacity isn't found (though it should be by now)
        cost_val = costs.get(f, {}).get(t, 'N/A')
        uij_val = arc_capacities.get(f, {}).get(t, 'N/A')
        lij_val = arc_lower_bounds.get(f, {}).get(t, 'N/A')
        print(f"  {f} -> {t}: {val:,.2f} Tons (Cost: {cost_val:,.2f} Rwfs/T, UIJ: {uij_val:,.2f} Tons, LIJ:
{lij_val:,.2f} Tons)")
    print("\nSupply Utilization:")
    for s in supply_districts:
        # Sum outgoing flows from supply nodes
        outgoing_flow = sum(flow[s, t].varValue for f_key, t in flow if f_key == s and flow[s, t].varValue
is not None)
        print(f"  {s}: {outgoing_flow:,.2f} / {supply_nodes.get(s, 0):,.2f} Tons supplied")
    print("\nDemand Fulfillment:")
    for d in demand_districts:
        # Sum incoming flows to demand nodes
        incoming_flow = sum(flow[f, d].varValue for f, t_key in flow if t_key == d and flow[f,
d].varValue is not None)
        print(f"  {d}: {incoming_flow:,.2f} / {demand_nodes.get(d, 0):,.2f} Tons demanded")
    print("\nWarehouse Throughput:")
    for w in warehouses:
        # Sum incoming and outgoing flows for warehouses
        incoming_flow_w = sum(flow[f, w].varValue for f, t_key in flow if t_key == w and flow[f,
w].varValue is not None)
        outgoing_flow_w = sum(flow[w, t].varValue for f_key, t in flow if f_key == w and flow[w,
t].varValue is not None)
        print(f"  {w}: In {incoming_flow_w:,.2f} Tons, Out {outgoing_flow_w:,.2f} Tons (Capacity:
{eax_warehouse_capacities.get(w, 0):,.2f} Tons)")
elif prob.status == -1: # Represents pulp.LpStatus.Infeasible
    print("The problem is Infeasible. There's no way to satisfy all demand given the current supply and
capacities.")
    print("Common reasons and troubleshooting steps:")
    print("1. **Total Supply vs. Total Demand:** Check the 'Initial Feasibility Check' above. If total
supply is less than total demand, the problem is impossible.")
    print("2. **Lower Bound (LiJ) Constraints:** These are often the cause of infeasibility. Try setting
`ENABLE_LIJ_CONSTRAINTS = False` at the top of the script and re-run.")
```

```
    print("   If it becomes feasible, then your LiJ values are too restrictive for the network's capabilities.")
    print("3. **Arc Capacities (UIJ):** Are certain arcs (paths) too small to carry the required flow? Look for bottlenecks.")
    print("4. **Warehouse Capacities:** Are warehouse throughput capacities too low to handle the transshipment volume?")
    print("5. **Network Connectivity:** Ensure every demand point has a viable path from a supply point, considering all capacities.")
elif prob.status == -3: # Represents pulp.LpStatus.Undefined
    print("The problem is Undefined. This can happen if the solver cannot find a solution (e.g., highly degenerate model) or if there are numerical issues.")
else:
    print(f"Problem could not be solved to optimality. Current Status: {pulp.LpStatus[prob.status]}")
```